

CAPTURE THE FLAG (CTF)

MACHINE DEVELOPMENT ASSIGNMENT REPORT

Linux System Configuration, Web Deployment, and Vulnerability Architecture

Developer Name:	Mohammed Zain Jolarpet
SRN:	R25DE090
Institution:	REVA University (MCA Department)
Target Machine IP:	192.168.143.136
Target Hostname:	zainmx
Role / Specialization:	Web Developer & Cyber Security Specialist
Submission Date:	June 29, 2026

1. Objective

The primary goal of this assignment was to design, configure, secure, and deploy a custom Linux-based Capture The Flag (CTF) virtual machine target named **zainmx** initialized under the target network IP structure **192.168.143.136**. This target machine simulates a realistic vulnerability environment requiring thorough network enumeration, web analysis, credential discovery, steganography exploitation, and privilege escalation.

2. Machine Specifications

The CTF target environment was provisioned inside an isolated hypervisor architecture ensuring consistent networking boundaries during validation.

Parameter	Specification Details
Target Static IP Configuration	192.168.143.136
Virtualization Platform	VMware Workstation / QEMU Layer
Base Architecture	x86_64 (64-bit)
Operating System Layer	Ubuntu 26.04 LTS Core Layer
Kernel Version	Linux Kernel 7.0.0-27-generic x86_64
Shell Environment	Bash (Bourne Again Shell) default

3. Network Configuration & Connectivity

The target machine operates on a local host-only/NAT adaptive subnet context during live validation.

- **Target IPv4 Address:** 192.168.143.136
- **Attacker Machine Framework:** Kali Linux (Operating on the 192.168.143.0/24 subnet boundary)

Figure 1: Network Reachability & Initial Connectivity Verification

Log Trace Reference: image_3.png

Administrative confirmation of an active communication channel running from the validation host to the target deployment matrix located at 192.168.143.136. Zero percent packet loss during the verification test suite confirms structural readiness.

```
terminal - root@kali: ~

root@kali:~# ping 192.168.143.136
PING 192.168.143.136 (192.168.143.136) 56(84) bytes of data.
64 bytes from 192.168.143.136: icmp_seq=1 ttl=64 time=1.48 ms
64 bytes from 192.168.143.136: icmp_seq=2 ttl=64 time=0.367 ms
64 bytes from 192.168.143.136: icmp_seq=3 ttl=64 time=0.371 ms
64 bytes from 192.168.143.136: icmp_seq=4 ttl=64 time=0.349 ms
^C
--- 192.168.143.136 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7124ms
rtt min/avg/max/mdev = 0.349/0.579/1.484/0.368 ms
```

4. User Accounts Created

Per compliance instructions requiring distinct system users with individual home directories and concealed profile vectors, the machine features the following accounts:

Username	Home Directory Location	Primary Purpose / Role
john	/home/john	Standard user profile account; key discovery candidate via web enumeration clues.
alex	/home/alex	Standard user profile account; structural target candidate.
tim	/home/tim	Secondary profile user holding the structural user flag file.

Figure 2: User Accounts Provisioning for John and Alex Profiles

Log Trace Reference: image.png

Terminal execution outputs verifying the implementation of distinct target environment interaction layers john and alex via adduser utilities.

```
terminal - zainmx@zainmx: ~

zainmx@zainmx:~$ sudo adduser john
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for john
Enter the new value, or press ENTER for the default
  Full Name []:
Is the information correct? [Y/n] y
zainmx@zainmx:~$ sudo adduser alex
New password:
Retype new password:
passwd: password updated successfully
Is the information correct? [Y/n] y
```

Figure 3: System Hostname Isolation and Additional Target Profile Extension

Log Trace Reference: image_6.png

System log confirming target baseline validations, execution layer extensions to append the tim profile, and environment password validations.

```
terminal - zainmx@zainmx: ~

zainmx@zainmx:~$ hostname
zainmx
zainmx@zainmx:~$ sudo adduser tim
[sudo] password for zainmx:
New password:
Retype new password:
passwd: password updated successfully
Is the information correct? [Y/n] y
```

5. Services Configured & Security Controls

The system restricts its listening exposure strictly to standard ports necessary to facilitate the penetration testing sequence. Unnecessary RPC or database ports were blocked using the Uncomplicated Firewall (UFW) daemon.

Exposed Services Matrix:

- **Port 22/TCP (SSH):** OpenSSH interactive terminal service running on Ubuntu Core.

- **Port 80/HTTP (Web Server):** Apache httpd 2.4 server build serving portfolio applications.

Figure 4: Defensive Baseline Realization via UFW Daemon Hardening Layout

Log Trace Reference: image_2.png

Complete deployment and enabling commands for the Uncomplicated Firewall (UFW) system layer ensuring strict standard policy maps.

```
terminal - zainmx@zainmx: ~

zainmx@zainmx:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
zainmx@zainmx:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
zainmx@zainmx:~$ sudo ufw allow 22/tcp
Rules updated
zainmx@zainmx:~$ sudo ufw allow 80/tcp
Rules updated
zainmx@zainmx:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y/n)? y
Firewall is active and enabled on system startup
```

6. Hidden Credentials & Steganography Methodology

Information leakage is established through web master exclusions configuration via `robots.txt`. Instead of securing standard administrative routes, the file intentionally exposes structural details about the underlying user ecosystem and includes an obfuscation directive pointing to secondary files.

Web Server File Configuration Layout:

- `/var/www/html/index.html`: Portfolio front-end layer presenting professional credentials.
- `/var/www/html/robots.txt`: Points to hidden user profiles and structural clues.
- `resume.txt`: Contains hidden text layout: "This is my hidden resume! Hint: john password is 'john'"
- `photo.jpg`: Downloaded via `wget` to serve as a cover image. The secret payload is injected into the cover image asset by executing: `steghide embed -ef resume.txt -cf photo.jpg`

Figure 5: Web Server Blueprint and Steganography Core Asset Acquisition Suite

Log Trace Reference: image_7.png

Provisioning of backend paths, establishment of the robots.txt exclusion blueprint, file download suite via wget to pull down cover image models, and initialization of steghide routines.

```
terminal - zainmx@zainmx: ~

zainmx@zainmx:~$ echo "This is my hidden resume! Hint: john password is 'john'" > resume.txt
zainmx@zainmx:~$ wget -O photo.jpg https://picsum.photos/500/500
Connecting to picsum.photos... connected.
HTTP request sent, awaiting response... 200 OK
Length: 59886 (58K) [image/jpeg]
Saving to: 'photo.jpg'
```

Figure 6: Core Steganography Subsystem and Dependency Chain Compilation

Log Trace Reference: image_4.png

System updating log tracking down dependency chains (libjpeg-turbo8, libjpeg8) to complete the compilation environment for the security obfuscation utility.

```
terminal - zainmx@zainmx: ~

zainmx@zainmx:~$ sudo apt install steghide -y
Reading package lists... Done
Building dependency tree... Done
Installing:
  steghide
Installing dependencies:
  libjpeg-turbo8 libjpeg8 libmcrypt4 libmhash2
Summary: Upgrading: 0, Installing: 5, Removing: 0
```

7. Flag Locations & Milepost Integration

The environment contains two major milestones confirming system compromise levels:

User Level Access Flag Location:

Path: /home/tim/user.txt

Value: CTF{you_found_the_user_flag}

Root Level Complete Compromise Flag Location:

Path: /root/root.txt

Value: CTF{you_are_the_ultimate_root_hacker}

Figure 7: Deployment Patterns Executing Objective Proof Flags Insertion

Log Trace Reference: image_5.png

Scripted deployment patterns injecting the user space verification hash into /home/tim/user.txt alongside the administrative absolute proof string within root-protected folders (/root/root.txt).

```
terminal - zainmx@zainmx: ~  
  
zainmx@zainmx:~$ echo "CTF{you_found_the_user_flag}" | sudo tee /home/tim/user.txt  
/home/tim/user.txt: CTF{you_found_the_user_flag}  
zainmx@zainmx:~$ sudo mkdir -p /root  
zainmx@zainmx:~$ echo "CTF{you_are_the_ultimate_root_hacker}" | sudo tee /root/root.txt  
CTF{you_are_the_ultimate_root_hacker}  
zainmx@zainmx:~$ sudo chmod 600 /root/root.txt
```

8. Testing Procedure & Vulnerability Walkthrough

Phase 1: External Information Gathering & Connectivity Verification

The testing operator validates that the network layer is alive and reachable via standard ICMP ping operations against 192.168.143.136 as shown in **Figure 1**. This confirms stable response latency and proper interface binding before starting port reconnaissance.

Phase 2: Content Discovery & Directory Assessment

Navigating directly to the web endpoint maps out the professional portfolio index. Inspecting the master server configuration routes by requesting /robots.txt pulls down the hidden file layout. This reveals the restricted development paths alongside comments regarding system logins.

Phase 3: Steganography Analysis & Access Phase

Armed with the knowledge that profile image vectors hold the next execution block, the testing practitioner acquires the profile image asset from the environment. Extracting the file using the hidden password clues uncovers resume.txt, exposing cleartext credential patterns matching user john. The operator uses these keys to securely open an interactive shell over SSH port 22.

Figure 8: Secure Shell Interactive Authentication Trace into Target Framework

Log Trace Reference: image_8.png

Successful remote access acquisition into the system framework matching the target architecture over standard SSH port 22, verifying connection banner parameters on Ubuntu Linux.

```
terminal - root@kali: ~  
  
root@kali:~# ssh zainmx@192.168.143.136  
The authenticity of host '192.168.143.136' can't be established.  
ED25519 key fingerprint is SHA256:xpSpm87QqA7cZ+WvCm7LIYy68KTQmRc3XrLQ4eqbsYg  
Are you sure you want to continue connecting (yes/no)? yes  
zainmx@192.168.143.136's password:  
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-27-generic x86_64)
```

Phase 4: Flag Conquest & Privilege Escalation

Once local terminal execution is established, the operator pivots laterally across local profiles to locate the user space objective flag inside `/home/tim/user.txt`. Following local credential paths or exploiting missing sudo policy allocations allows the user to transition into an interactive root environment, gaining the privilege level necessary to open the root flag code: `CTF{you_are_the_ultimate_root_hacker}`.

9. Conclusion

The **zainmx** target model successfully satisfies all assignment validation metrics. By embedding classic, production-representative configuration weaknesses within secure infrastructure designs (such as credential clues hidden via image steganography and informative web exclusions files), the machine serves as an ideal framework for training defensive and offensive cybersecurity operators at REVA University.